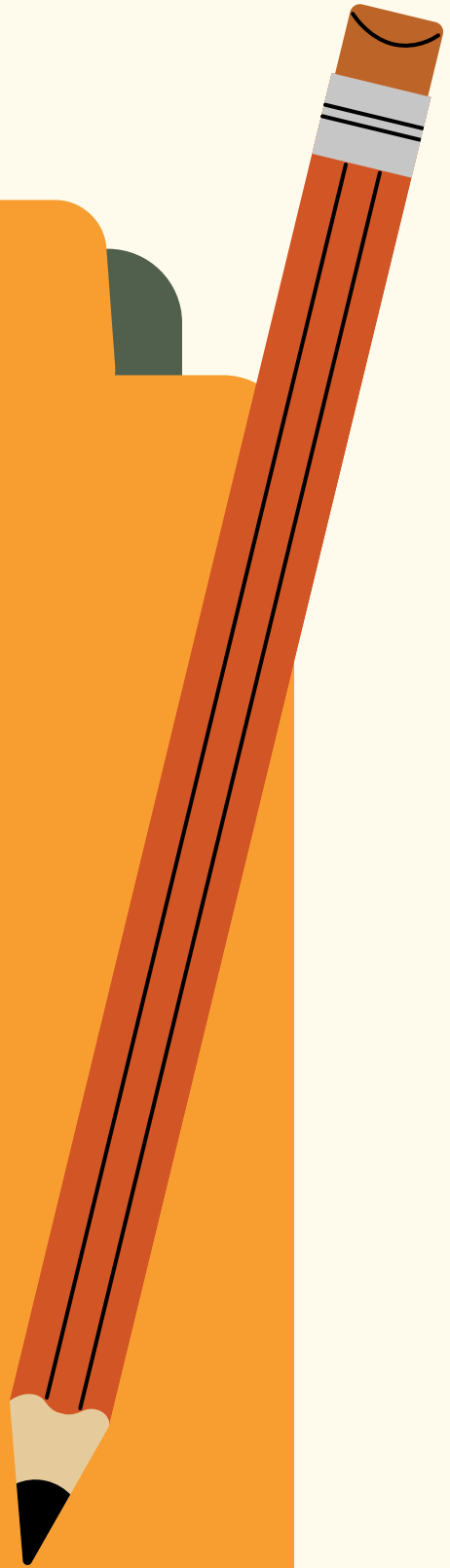


From numbers to AI

PROGRAMMING

파이썬





질문있나요?

<https://discord.gg/mQa86T9NU2>

cm@playidealab.com 또는 lcmin@yonsei.ac.kr

모르는건 당연한거지만, 모르는채로 있는건 선택이지요

질문하는 과정에서 깨닫기도 합니다



CLASS



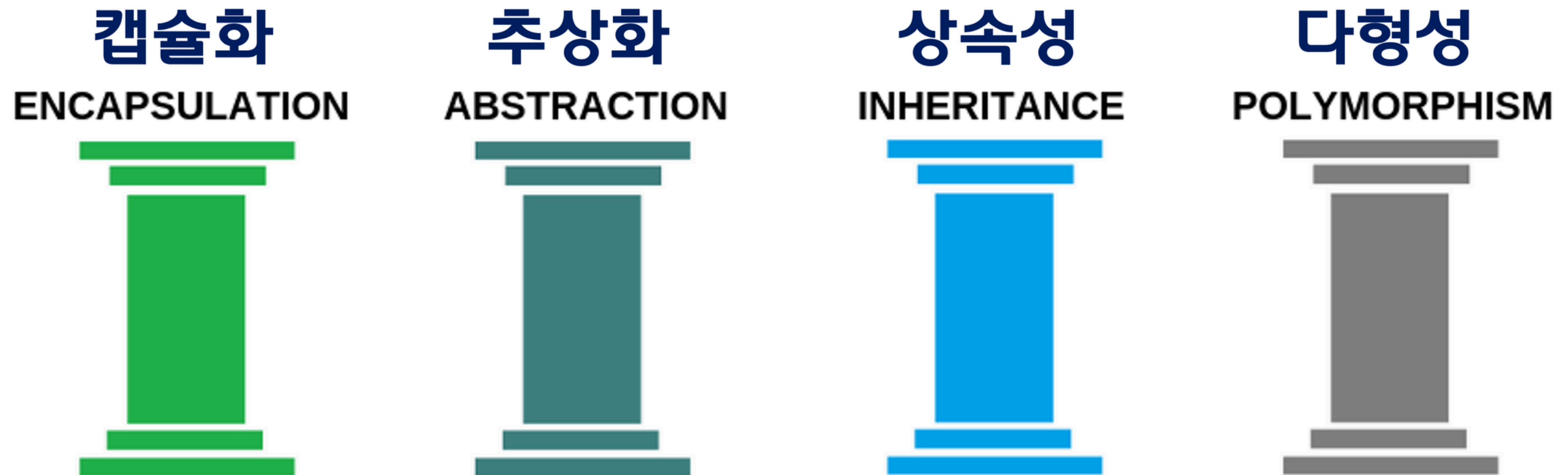
CLASS



- class를 활용해 원하는 기능을 구현할 수 있다
- 파이썬이 class로 구성되어 있음을 이해한다
- class를 활용해 파이썬의 기능을 일부 재현, 혹은 수정할 수 있다

Object Oriented Programming

The Four Pillars of OOP

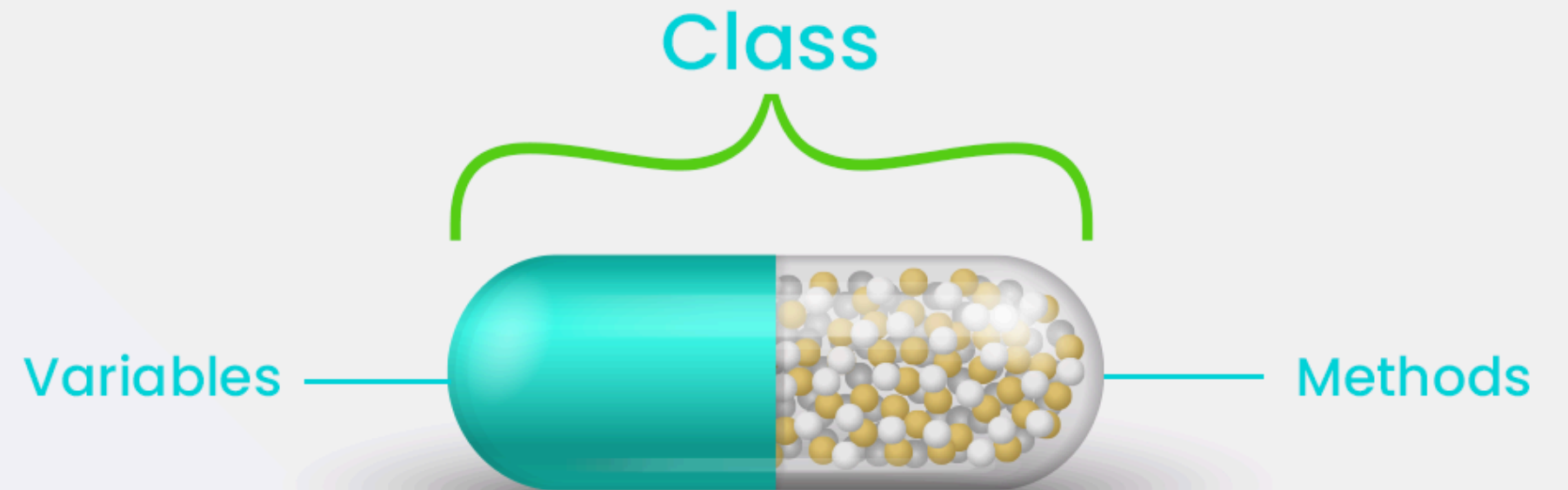


Encapsulation

“하나의 객체 = 기능 + 요소”

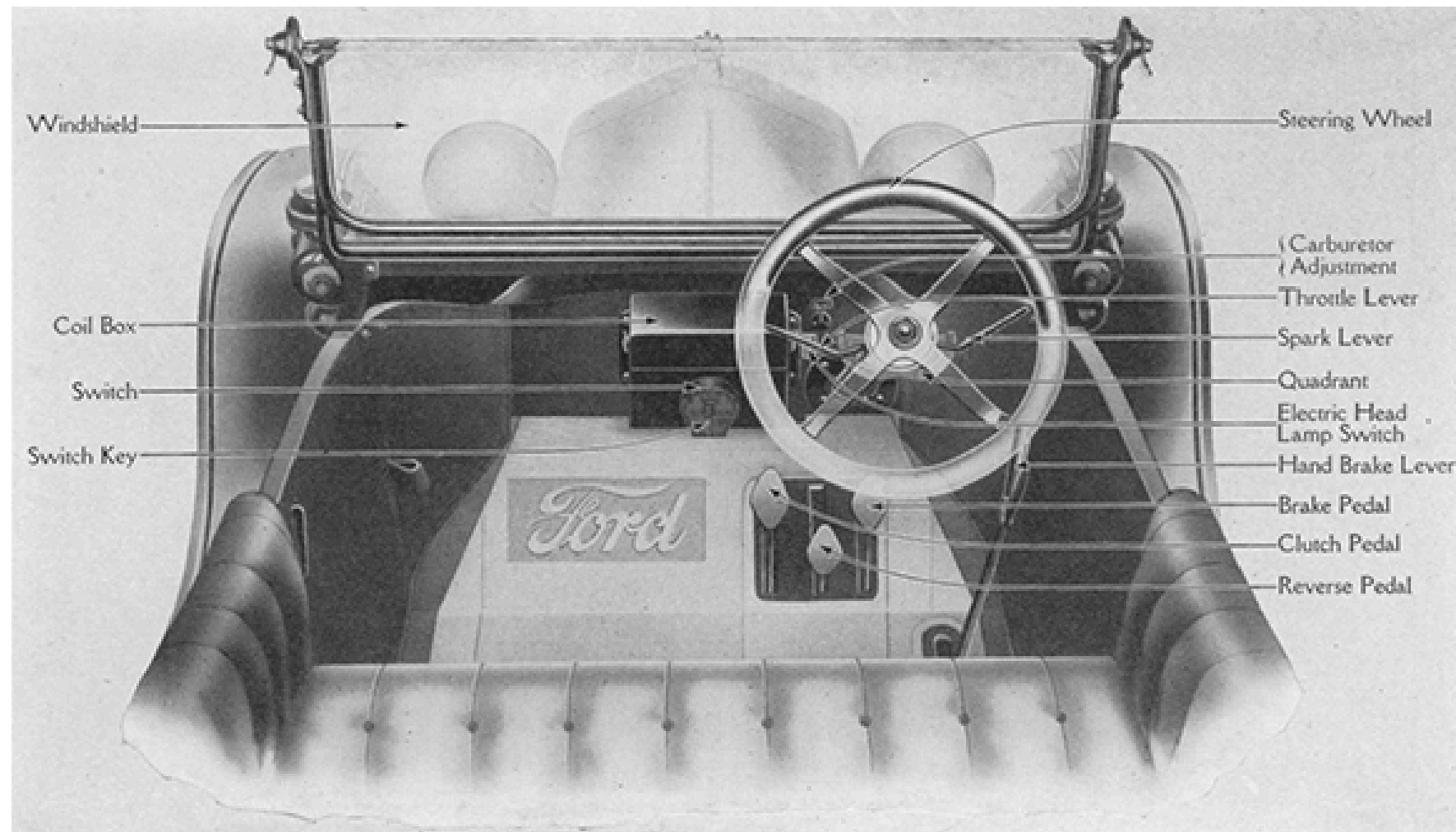
```
class Cat:
    """ 고양이 클래스
    입력:
    - name: (str) 고양이의 이름입니다.
    - color: (str) 고양이 색상입니다.
    - age: (int) 고양이 나이입니다.
    """
    def __init__(self, name, color, age):
        self.name = name
        self.color = color
        self.age = age

    def introduce_myself(self):
        print(f"내 이름은 {self.name}, {self.color}이고, {self.age}살이야 ")
```



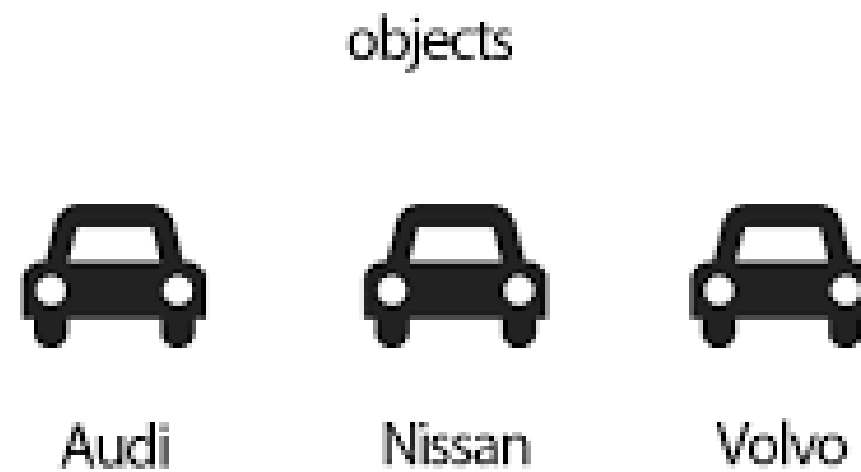
Abstraction

“구현의 세부 정보는 노출되지 않고, 인터페이스 만을 활용”



만약.. 내부의 구현을 바꾼다 하더라도 인터페이스는 바뀌지 않음

Inheritance



“반복되는 요소는 재사용”

```
class Human:
    population = 0

    def __init__(self, name):
        self.name = name
        self.age = 0
        self.job = None
        Human.population += 1

    def introduce(self):
        print(self.name, self.age, self.job)
```

```
class NPC(Human):
    def __init__(self, name, quest):
        super().__init__(name)
        Human.population += 1

        self.job = "NPC"
        self.HP = 9999
        self.quest = quest

    def get_info(self):
        print(self.quest)
```

```
class Swordman(Human):
    def __init__(self, name):
        super().__init__(name)
        self.job = "swordman"
        self.weapon = None
        self.HP = 100

    def get_weapon(self, weapon):
        self.weapon = weapon

    def attack(self, target):
        print(f"{self.weapon.name}를 휘두른다!")
        print(f"{self.weapon.dmg}만큼의 피해!")
        target.attacked(self.weapon.dmg)
```

Polymorphism



“같은 기능이더라도 목적에 따라..
다양한(poly) 형태(shape=morph)를 가질 수 있음”

Overriding: 같은 기능을 재정의 하는 것
부모가 return을 정해줌.. Return은 같아야 한다.

```
class Hello:
    def say_hello(self):
        print("Hello")
```

```
class 안녕(Hello):
    def say_hello(self):
        print("안녕")
```

```
class 곤니찌와(Hello):
    def say_hello(self):
        print("곤니찌와")
```

```
hi = 안녕()
hi.say_hello()
```

```
hi2 = 곤니찌와()
hi2.say_hello()
```

[2] ✓ 0.3s

... 안녕
곤니찌와

Class란?

number, str, list, tuple, set, dict...
알고 보니 모두 class?

CLASS

- 파이썬에서

- $1 + 2 = 3$
- `"1" + "2" = "12"`
- Type이 다르기 때문
- Type마다 정해진 기능 有
- 이 type들은 파이썬에서 만들어서 미리 구현해 둔 class들
- `help("알아보고 싶은 type")`을 실행해 보자

```
▶ print(1 + 2)
   print("1" + "2")

3
12
```

```
▶ print(type(1), type(2))
   print(type("1"), type("2"))

<class 'int'> <class 'int'>
<class 'str'> <class 'str'>
```

CLASS

- 빌트인 모듈인 class int
- Int의 사칙연산 등의 기능은 모두 Int Class 안 함수들, 즉 method로 구현된 것들

```
[11] num1 = 2
    num2 = 3
    print(num1.__add__(num2))    # +
    print(num1.__sub__(num2))    # -
    print(num1.__mul__(num2))    # *
    print(num1.__le__(num2))     # less or equal
    print(num1.__ge__(num2))     # greater or equal
    print(num1.__eq__(num2))     # equal

5
-1
6
True
False
False
```

help(int)

Help on class int in module builtins:

```
class int(object)
|   int([x]) -> integer
|   int(x, base=10) -> integer
|
|   Convert a number or string to an integer, or return 0 if no arguments
|   are given.  If x is a number, return x.__int__().  For floating point
|   numbers, this truncates towards zero.
|
|   If x is not a number or if base is given, then x must be a string,
|   bytes, or bytearray instance representing an integer literal in the
|   given base.  The literal can be preceded by '+' or '-' and be surrounded
|   by whitespace.  The base defaults to 10.  Valid bases are 0 and 2-36.
|   Base 0 means to interpret the base from the string as an integer literal.
|   >>> int('0b100', base=0)
|   4
|
|   Methods defined here:
|
|   __abs__(self, /)
|       abs(self)
|
|   __add__(self, value, /)
|       Return self+value.
|
|   __and__(self, value, /)
|       Return self&value.
|
|   __bool__(self, /)
|       self != 0
```

CLASS

- 빌트인 모듈인 class list
- 더하기, 대소 비교, 삭제, 곱하기 등의 메서드가 정의되어 있다

```
a_list = [1, "1"]  
print(a_list.__add__([1, 2, 3]))  
print(a_list)
```

```
[1, '1', 1, 2, 3]  
[1, '1']
```

```
[8] a = [1, "1"]  
print(a.__iadd__("2"))  
print(a)  
a.__iadd__("3")  
print(a)
```

```
[1, '1', '2']  
[1, '1', '2']  
[1, '1', '2', '3']
```

help(list)

Help on class list in module builtins:

```
class list(object)  
| list(iterable=(), /)  
|  
| Built-in mutable sequence.  
|  
| If no argument is given, the constructor creates a new empty list.  
| The argument must be an iterable if specified.  
|  
| Methods defined here:  
|  
| __add__(self, value, /)  
|     Return self+value.  
|  
| __contains__(self, key, /)  
|     Return key in self.  
|  
| __delitem__(self, key, /)  
|     Delete self[key].  
|  
| __eq__(self, value, /)  
|     Return self==value.  
|  
| __ge__(self, value, /)  
|     Return self>=value.  
|  
| __getattr__(self, name, /)  
|     Return getattr(self, name).  
|  
| __getitem__(...)  
|     x.__getitem__(y) <=> x[y]  
|  
| __gt__(self, value, /)  
|     Return self>value.  
|  
| __iadd__(self, value, /)  
|     Implement self+=value.  
|  
| __imul__(self, value, /)  
|     Implement self*=value.  
|  
| __init__(self, /, *args, **kwargs)  
|     Initialize self. See help(type(self)) for accurate signature.
```

CLASS

- 빌트인 모듈인 class str
- "1" * 5의 결과는 11111, c에서는 안 된다..__mul__ method가 정의되어 있기 때문에 가능한 일

```
my_str = "Python_Test"
my_str2 = "Assembly"
my_str3 = "pyyyyypppypppppppppttttthhhhoooooooooonnnn"
print(my_str.__eq__("Python_Test"))    # my_str == ~
print(my_str.__eq__(my_str2))          # my_str == ~
print(my_str2.count("s", 1, 2))         # 요소 갯수를 셀 때 시작 인덱스와 끝 인덱스를 넣어줘서 구간
print(my_str.replace("Test", "Project")) # 요소를 변경해준다
print(my_str.replace("_Test", ""))      # 요소를 빈칸과 교환함으로써 제거하는 것도 가능하다
print(my_str3.replace("p", "", 1))      # 몇 개를 제거할 지 정할 수 있다
print(my_str3.replace("p", "", 3))      |
print(my_str3.replace("p", ""))         # 숫자를 입력해주지 않으면 모두 제거한다
```

```
True
False
1
Python_Project
Python
yyyyyypppypppppppppttttthhhhoooooooooonnnn
yyyyyyyypppppppppppttttthhhhoooooooooonnnn
yyyyyyyytytttthhhhoooooooooonnnn
```

help(str)

Help on class str in module builtins:

```
class str(object)
| str(object='') -> str
| str(bytes_or_buffer[, encoding[, errors]]) -> str

| Create a new string object from the given object. If encoding or
| errors is specified, then the object must expose a data buffer
| that will be decoded using the given encoding and error handler.
| Otherwise, returns the result of object.__str__() (if defined)
| or repr(object).
| encoding defaults to sys.getdefaultencoding().
| errors defaults to 'strict'.

| Methods defined here:

| __add__(self, value, /)
|     Return self+value.

| __contains__(self, key, /)
|     Return key in self.

| __eq__(self, value, /)
|     Return self==value.

| __format__(self, format_spec, /)
|     Return a formatted version of the string as described by format_spec.

| __ge__(self, value, /)
|     Return self>=value.

| __getattr__(self, name, /)
|     Return getattr(self, name).

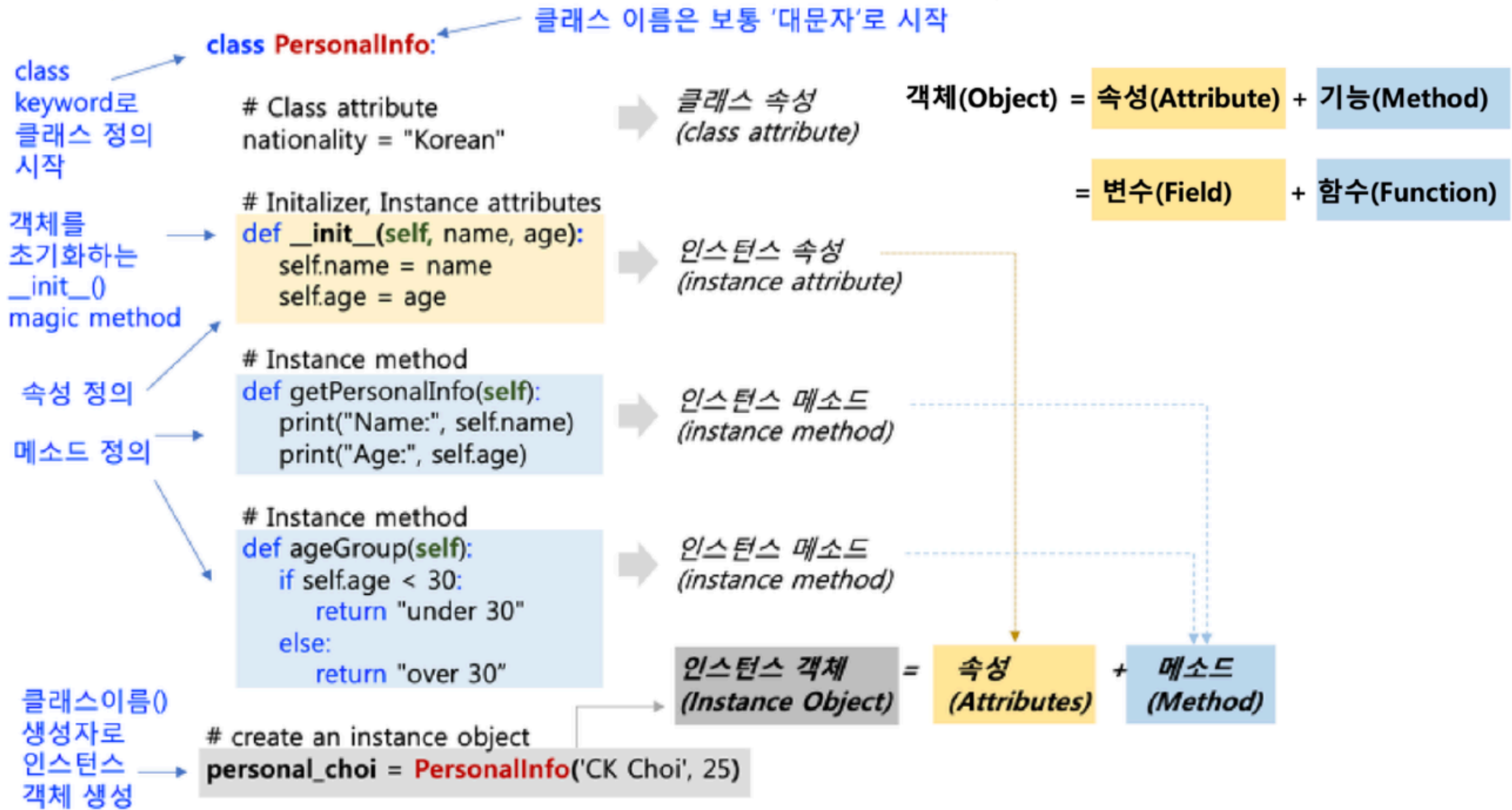
| __getitem__(self, key, /)
|     Return self[key].

| __getnewargs__(...)

| __gt__(self, value, /)
|     Return self>value.
```

Class of Python

- 그래서 클래스를 왜 쓰나요? :: 원하는 type을 만들어서 쓰는 것
- 클래스는 객체를 생성하는 틀
 - 어떤 속성과 기능을 가진 객체를 많이 만들고 싶다면?
 - 예를 들어 타고 다닐 수 있다는 기능과 차 번호, 차종 등의 속성을 지닌 차라는 객체를 계속 만들어야 한다면?
 - 방법1: 매번 차의 기능을 새로 설계하고 차 번호판도 매번 새로 디자인 해서 만든다
 - 방법2: 차와 차 번호판의 틀을 만들어 두고 필요에 따라 조금씩 변형하여 여러 차를 만든다
 - 방법3: CLASS 쓰기
 - 문자열을 다루기 위해 매번 str의 메서드들을 정의하고 속성을 부여해야 하는 상황을 상상해 봅시다...



Class의 개념

- instance
 - 객체
 - Method
 - self
- 앞서 str, int, list, tuple, set, dict 등 자료형
 - Class
 - a = str(1)
 - a :: str class의 instance
 - For instance라는 표현을 떠올려보자
 - Method: class 내의 함수

Class의 개념

- instance
- 객체
- Method
- self

○ Self는 왜 계속 넣어줘야 하나요? – 넣지 말아 보자

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def get_age(self):
        return f"개의 나이: {self.age}살"

    def set_age(self, age):
        self.age = age

    def set_name(self, name):
        self.name = name
```

```
d = Dog("ninja", 5)
d.get_age()
```

'개의 나이: 5살'

```
def get_age():
    return f"개의 나이: {self.age}살"

def set_age(self, age):
    self.age = age

def set_name(self, name):
    self.name = name
```

```
d = Dog("ninja", 5)
d.get_age()
```

```
-----
TypeError                                Traceback (most recent call last)
<ipython-input-4-8655e8756db8> in <module>()
     31
     32 d = Dog("ninja", 5)
--> 33 d.get_age()
     34
```

TypeError: get_age() takes 0 positional arguments but 1 was given

Class의 개념

- instance
 - 객체
 - Method
 - self
- Self는 왜 계속 넣어줘야 하나요? – 넣지 말아 보자
 - 그랬더니 앞서 확인한 것처럼 get_age() 함수는 positional argument를 0개 받는데 1개가 주어졌다며 에러가 뜬다
 - 괄호 안에 아무것도 안 넣었는데..?
 - d.get_age()에서 d가 인자로 넘겨지기 때문
 - instance 이름". "method or instance 변수"와 같이 method 나 instance 변수에 접근
 - self가 없으면 ~.get_age() 같은 식으로 메서드를 실행시킬 수 없다.

Class의 개념

○ Self의 정체

- id 함수를 통해 확실히 하자
- 인스턴스 d의 주소 값과 d의 self의 주소 값이 동일
- Class 내의 self는 class instance다

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def get_age(self):
        return f"개의 나이: {self.age}살"

    def set_age(self, age):
        self.age = age

    def set_name(self, name):
        self.name = name

    def id_print(self):
        print(f"self의 주소: {id(self)}")
```

```
d = Dog("ninja", 5)
d.id_print()
print(f"d의 주소: {id(d)}")
```

```
self의 주소: 140647930036112
d의 주소: 140647930036112
```

Class의 개념

- 'Class 이름.메서드'로 method 호출
 - 아까 만든 Dog class에 다음과 같은 method 추가 (self 없이)
 - Method 매개 변수에 Self가 없어도 class명을 사용하는 경우 method 호출이 가능하다

```
def using_classname():  
    print("클래스 이름을 활용한 호출 예시")
```

```
Dog.using_classname()
```

```
클래스 이름을 활용한 호출 예시
```

Class의 개념

- 'Class 이름.메서드'로 method 호출
 - 반대로 Method 매개 변수에 Self가 있다면?
 - Self 인자가 없다고 에러 발생
 - 해결법: instance를 하나 생성해서 method 인자로 넣어준다(self = instance 자체)

```
Dog.id_print()
```

```
-----  
TypeError                                Traceback (most recent call last)  
<ipython-input-3-cf8c91960183> in <module>()  
----> 1 Dog.id_print()
```

```
TypeError: id_print() missing 1 required positional argument: 'self'
```

```
jindo_mix = Dog("jangbi", 3)
```

```
Dog.id_print(jindo_mix)
```

```
self의 주소: 140035684416976
```

Method의 종류

- Instance method
- Class method
- Static method
- Instance method
 - 가장 기본적인 메서드
 - 아무 데코레이터(ex-@classmethod) 없이 선언하면 인스턴스 메서드로 취급됨
 - 첫번째 매개 변수 self가 클래스의 인스턴스를 인자로 받는다
 - 클래스 메서드, 정적 메서드와 달리 반드시 먼저 인스턴스를 생성한 후 호출해야 한다

Method의 종류

- Instance 메서드의 예시

```
class Counter:

    def __init__(self, value = 0):
        self.value = value

    def increment(self, delta = 1):
        self.value += delta
        return self.value

    def decrement(self, delta = 1):
        self.value -= delta
        return self.value
```

```
counter1 = Counter()
print(counter1.value)
print(counter1.increment(3))
print(counter1.decrement(2))
```

```
0
3
1
```

Calc를 instance 메서드로 구현해 보면?

```
class Calc:

    def __init__(self, a, b):
        self.a = a
        self.b = b

    def add(self):
        print(self.a + self.b)

    def mul(self):
        print(self.a * self.b)

    def add_one(self):
        self.a += 1
        self.b += 1
        print(self.a, self.b)
```

```
Calc_1 = Calc(2, 3)
Calc_1.add()
Calc_1.mul()
print(Calc_1.a, Calc_1.b)
```

```
5
6
2 3
```

Method의 종류

- Instance method
- Class method
- Static method
- class method
 - @classmethod와 같이 사용
 - 인스턴스 없이 호출 가능한 점이 정적 메서드와 유사한 점
 - 첫번째 매개변수에 클래스의 인스턴스가 아닌 class 자체(cls)가 넘어오는 것이 인스턴스 메서드와의 차이점
 - 객체의 속성/메서드에는 액세스가 불가능
 - cls.클래스속성명으로 클래스 속성에는 접근 가능

Method의 종류

- Class 메서드의 예시

```
class User:
    def __init__(self, email, password):
        self.email = email
        self.password = password

    @classmethod
    def fromTuple(cls, tup):
        return cls(tup[0], tup[1])

    @classmethod
    def fromDictionary(cls, dic):
        return cls(dic["email"], dic["password"])

user = User("user@test.com", "1234")
user.email, user.password

('user@test.com', '1234')
```

```
class User:
    def __init__(self, email, password):
        self.email = email
        self.password = password

    @classmethod
    def fromTuple(cls, tup):
        return cls(tup[0], tup[1])

    @classmethod
    def fromDictionary(cls, dic):
        return cls(dic["email"], dic["password"])

user = User("user@test.com", "1234")
user.email, user.password

('user@test.com', '1234')

user = User.fromDictionary({"email": "user@test.com", "password": "1234"})
user.email, user.password

('user@test.com', '1234')
```

만약 class 메서드가 없다면?

```
class User2:
    def __init__(self, email=None, password=None, gen_from_str=None, gen_from_tuple=None, gen_from_dictionary=None):
        if gen_from_str:
            user = gen_from_str.split(',')
            self.email = user[0]
            self.password = int(user[1])

        elif gen_from_dictionary:
            user = gen_from_dictionary
            self.email = user['email']
            self.password = int(user['password'])

        elif gen_from_tuple:
            self.email = gen_from_tuple[0]
            self.password = gen_from_tuple[1]

        else:
            self.email = email
            self.password = password
```

```
user = User2(gen_from_str='~~@gmail.com, 1234')
user.email, user.password
```

```
('~~@gmail.com', 1234)
```

Method의 종류

- Instance method
- Class method
- Static method
- Class method
 - @classmethod 없이는 모든 매개변수의 기본값이 None
 - 새로운 인자를 받고자 한다면 또다시 매개변수에 None값을 주며 한 줄이 계속해서 길어짐
 - 함수 안에서 분기가 추가적으로 이루어짐

Method의 종류

- Instance method
- Class method
- Static method
- Static method
 - @staticmethod와 같이 사용
 - 첫번째 매개변수에 class나 인스턴스가 인자로 넘겨지지 않음
 - 인스턴스 속성에 접근 불가
 - 순수 함수 같은 유틸리티 메서드 구현에 많이 사용됨
 - 인스턴스의 내용과는 상관없이 결과만 구하면 될 때!

Method의 종류

```
# 유틸리티 메서드 구현에 자주 사용되는 static method
```

```
class StringUtils:
    @staticmethod
    def toCamelcase(text):
        words = iter(text.split("_"))
        return next(words) + "".join(i.title() for i in words)

    @staticmethod
    def toSnakecase(text):
        letters = ["_" + i.lower() if i.isupper() else i for i in text]
        return "".join(letters).lstrip("_")
```

```
StringUtils.toCamelcase("last_modified_date")
```

```
'lastModifiedDate'
```

```
StringUtils.toSnakecase("lastModifiedDate")
```

```
'last_modified_date'
```

Method의 종류

```
# __init__ 없이, self 없이, instance 메서드 사용 불가 - static 메서드
class Calc:
    @staticmethod
    def add(a, b):
        print(a + b)

    @staticmethod
    def mul(a, b):
        print(a * b)
```

```
Calc.add(10, 20)    # 클래스에서 바로 메서드 호출
Calc.mul(10, 20)    # 클래스에서 바로 메서드 호출
```

```
30
200
```

```
# Calc를 instance 메서드로 구현해 보면?
class Calc:
```

```
    def __init__(self, a, b):
        self.a = a
        self.b = b
```

```
    def add(self):
        print(self.a + self.b)
```

```
    def mul(self):
        print(self.a * self.b)
```

```
    def add_one(self):
        self.a += 1
        self.b += 1
        print(self.a, self.b)
```

```
Calc_1 = Calc(2, 3)
Calc_1.add()
Calc_1.mul()
print(Calc_1.a, Calc_1.b)
```

```
5
6
2 3
```

Class of Python

- 생성자 `__init__` 왜 쓰나?
 - 까먹고 초기 값을 안 설정해주면 에러 발생
 - `__init__`에 초기 값 설정을 해두면 instance 선언 시 초기 값을 까먹고 안 넣으면 instance 선언 자체가 안 됨

```
a = MyStrAdd()
a.add_()

-----
AttributeError                                Traceback (most recent call last)
<ipython-input-45-2fd15f5baa2d> in <module>()
      1 a = MyStrAdd()
----> 2 a.add_()

<ipython-input-43-63cde4810662> in add_(self)
      5
      6     def add_(self):
----> 7         return self.str1 + self.str2

AttributeError: 'MyStrAdd' object has no attribute 'str1'
```

Class 네임스페이스

- Class 네임스페이스
- Instance 변수
- Class 변수
- 복습: 네임스페이스
 - 해당 범위에 해당하는 변수들의 이름과 값이 dict로 저장
 - Class에도 네임스페이스가 있다
 - Locals를 실행해 보면 class 내부의 local 변수인 family에 'Canidae'라는 값이 들어간 걸 확인할 수 있다

```
def local_check():  
    a = 1  
    b = 'CM'  
    print(locals())
```

```
local_check()
```

```
{'a': 1, 'b': 'CM'}
```

```
class Doggy:  
    family = 'Canidae'           # 개과(科) 동물이다  
    print(locals())  
    def __init__(self, name, age):  
        self.name = name  
        self.age = age
```

```
{'__module__': '__main__', '__qualname__': 'Doggy', 'family': 'Canidae'}
```

Class 네임스페이스

- Instance를 선언해보면 네임스페이스에 추가되어 있는 걸 확인할 수 있다

```
d = Doggy('ninja', 5)
print(globals())
```

```
'd': <__main__.Doggy object at 0x7f74296ad990>,
```

- `__dict__` method로 instance의 네임스페이스를 간편하게 확인 가능

```
d = Doggy('ninja', 5)
print(d.__dict__)
```

```
{'name': 'ninja', 'age': 5}
```

Class 네임스페이스

- 그런데...class 변수인 family는 ninja의 네임스페이스에 없었는데 왜?
 - Instance의 네임스페이스에 해당 변수가 있는지 확인 후
 - 없으면 해당 instance의 class로 가서 다시 그 변수를 찾음
 - => class 변수

```
class Doggy:
    family = 'Canidae'
    # print(locals())
    def __init__(self, name, age):
        self.name = name
        self.age = age
```

```
ninja = Doggy("ninja", 5)
# print(locals())
```

```
ninja.family
```

```
'Canidae'
```

Class 네임스페이스

- Instance 변수는 해당 instance에만 적용

```
ninja.family = "우리 가족"  
print(f"ninja.family: {ninja.family}")
```

```
dd = Doggy("dd", 3)  
print(f"dd.family: {dd.family}")
```

```
ninja.family: 우리 가족  
dd.family: Canidae
```

예제

<https://blockdmask.tistory.com/443>

```
1 # 바나나폰 클래스
2 class BananaPhone:
3     phone_name = '어른폰'
4
5     def __init__(self, number, owner):
6         self.number = number
7         self.owner = owner
8
9     def print(self):
10         s = f'이 폰은 {self.phone_name} 기종이며\n'
11         s += f'번호는 {self.number} 이고\n'
12         s += f'이 폰의 주인은 {self.owner} 입니다.\n'
13         print(s)
14
15 # 객체, 인스턴스 생성
16 p1 = BananaPhone('010-1111-2222', '개님')
17 p2 = BananaPhone('010-7777-0000', '고양이님')
18
19 p1.print()
20 p2.print()
```

클래스 변수

메소드

인스턴스 변수

이 폰은 어른폰 기종이며
번호는 010-1111-2222 이고
이 폰의 주인은 개님 입니다.

이 폰은 어른폰 기종이며
번호는 010-7777-0000 이고
이 폰의 주인은 고양이님 입니다.

Class of Python

- 그렇다면 class를 한번 만들어 봅시다
 - 게임에서 마법사 직업군을 만들어야 한다고 가정
 - 2차 전직을 하면 기존 스킬은 그대로인 상태에서 몇몇 스킬만 추가되게 하고 싶다면?
 - 상속! - 레고처럼 작게 나뉜 클래스들을 모아모아 원하는 걸 만들어 보자

```
class Magician:

    def __init__(self):
        self.class_name = '마법사'
        self.main_ability = 'int'

    def energy_ball(self):
        print("에너지볼!")
        # hp가 있는 몬스터 class를 만들고 몬스터의 hp가 깎이게 할 수도?
```

```
class FlameWizard(Magician):

    def fire_ball(self):
        print("파이어볼!")
```

```
PLAYER_1 = FlameWizard()
PLAYER_1.fire_ball()
PLAYER_1.energy_ball()
```

파이어볼!
에너지볼!

```
class ThunderWizard(Magician):

    def thunder_bolt(self):
        print("썩더 볼트!")
```

```
PLAYER_2 = ThunderWizard()
PLAYER_2.energy_ball()
PLAYER_2.thunder_bolt()
```

에너지볼!
썩더 볼트!

Class of Python

- 2차 전직을 하면 그냥 에너지 볼 말고 강화된 에너지 볼을 쏘게 만들고 싶다면?
- 메서드 오버라이딩!
 - 부모 클래스(상속해줌)의 메서드와 같은 이름의 메서드를 자식 클래스(상속 받음)에 만들어 주면 자식 클래스의 메서드가 호출된다.

```
class FlameWizard(Magician):  
    def energy_ball(self):  
        print("강화된 에너지볼!")  
  
    def fire_ball(self):  
        print("파이어볼!")
```

```
PLAYER_1 = FlameWizard()  
PLAYER_1.fire_ball()  
PLAYER_1.energy_ball()
```

```
파이어볼!  
강화된 에너지볼!
```

Class of Python

- 클래스 변수의 활용: 모든 instance가 공유해야 하는 값이 있을 때

```
class Magician:
    num_magicians = 0

    def __init__(self):
        self.class_name = '마법사'
        self.main_ability = 'int'
        Magician.num_magicians += 1

    def energy_ball(self):
        print("에너지볼!")
        # hp가 있는 몬스터 class를 만들고

    def __del__(self):
        Magician.num_magicians -= 1
```

```
PLAYER_00 = Magician()
print(PLAYER_00.num_magicians, Magician.num_magicians)
PLAYER_01 = Magician()
print(PLAYER_01.num_magicians, Magician.num_magicians)
del PLAYER_00
print(Magician.num_magicians)
del PLAYER_01
print(Magician.num_magicians)
```

mod2 ×

C:\Users\ABC\PycharmProjects\pythonProject2\venv\Script

1 1

2 2

1

0

* 상속이 아닌 방법으로 기존 class 재활용하기

Composition

두 개의 별도의 클래스

```
class Calc:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
    def add(self):
        return self.x + self.y
```

```
    def subtract(self):
        return self.x - self.y
```

```
class Calc2:
    def __init__(self, x, y):
        self.x = x
        self.y = y
```

```
    def add(self):
        return self.x + self.y
```

```
    def multiply(self):
        return self.x * self.y
```

```
class Calc:
    def __init__(self, x, y):
        self.x = x
        self.y = y
        self.calc2 = Calc2(x, y) # 해당 클래스의 객체를 명시적으로 가져옴
```

```
    def add(self):
        return self.x + self.y
```

```
    def subtract(self):
        return self.x - self.y
```

```
    def multiply(self):
        return self.calc2.multiply() # 해당 클래스의 객체에 있는 메서드를 명시적으로 활용함
```

내부에 그대로 들고오자! = 포함!

해당 클래스의 기능을 그대로 사용!

```
calc1 = Calc(1, 2)
print(calc1.multiply())
calc2 = Calc(x=2, y=3) # 함수 인자와 마찬가지로 직접 인자명과 값을 써도 됨
print(calc2.multiply())
```

2
6

Multiply method를 가져다 쓰고 싶은데..

Class of Python

- Str끼리 빼기가 안 돼서 불편하다면?
- 만들어 보자

```
f = MyStr()
f.str1 = "abcde"
f.str2 = "cde"
f.subtract(f.str1, f.str2)
```

```
'ab'
```

```
aa = MyStr()
aa.str1 = " a"
aa.strip()
```

```
'a'
```

```
class MyStr:
    def __init__(self):
        self.str1 = ""
        self.str2 = ""
        # self.list1 = list1

    def add(self):
        result = self.str1 + self.str2
        return result

    def multiply(self, num):
        result = self.str1 * num    # , self.st
        return result

    def strip(self):
        clean_str1 = ""
        for i in range(len(self.str1)):
            if self.str1[i] != " ":
                clean_str1 += self.str1[i]

        return clean_str1

    def subtract(self, str1, str2):    # 원래 s
        result = ""
        for i in range(len(self.str1)):
            if self.str1[i] not in self.str2:
                result += self.str1[i]
        return result
```

추상 클래스

- 캐릭터가 hp, 공격력 등을 공통적으로 갖고 있게 하고 싶다면
- 상속!
- 만약 데미지를 받지 않는 캐릭터는 생성되지 못하게 하고 싶다면?
= 데미지를 받는 메서드의 구현을 강제하고 싶다면
- 추상 클래스를 사용
 - => 추상 클래스인 부모 클래스의 메서드를 오버라이딩하지 않으면 인스턴스 생성이 안 되게 제한함
 - 추상 클래스는 클래스 내부 메서드들의 내용이 전부 비어 있고 이름만 정의되어 있음

추상 클래스

Weapon Class(추상 클래스)

#공격하기 추상 메서드

Def 공격하기():

어떤 무기냐에 따라 다르잖아?
그래서 내용은 없어~(**pass**)

#방어하기 추상 메서드

def 방어하기()

공격과 마찬가지로 하위 클래스
에서 자세히 쓰자~(**pass**)

주의사항

상속받으면 모든 추상 메서드를 재정의할 것!!

Sword class(Weapon class 상속)

Def 공격하기():

찌르기
베기
던지기

Def 방어하기()

칼로 막기

Gun class(Weapon class 상속)

Def 공격하기():

총 쏘기
장전하기

Def 방어하기():

총으로 막기

< 추상 클래스에서 정의 된 추상 메서드는 하위 클래스에서 모두 재정의되어야 한다.>

추상 클래스

```
from abc import * # Abstrace Base Class

class Character(metaclass = ABCMeta):
    def __init__(self):
        self.hp = 100
        self.attack_power = 20

    def attack(self, other, attack_kind):
        other.get_damage(self.attack_power, attack_kind)

    @abstractmethod # Character 클래스를 상속받는 모든 클래스는 하기 함수를 오버라이딩으로 구현해야 인스턴스 생성이 가능
    def get_damage(self, attack_power, attack_kind):
        pass

# 테스트코드
# if __name__ == '__main__':
#     chl = Character() # TypeError 발생 (Can't instantiate abstract class Character with abstract methods get_damage)
```

추상 클래스

위의 경우에만 인스턴스 생성 가능

```
class Player(Character):  
    def get_damage(self, attack_power, attack_kind):  
        self.hp -= attack_power  
  
char1 = Player()
```

```
class Player2(Character):  
    pass  
  
char2 = Player2() # TypeError: Can't instantiate abstract class Player2 with abstract methods get_damage
```